

Continuous Implementation/ Continuous Delivery

Evolve from puppet to puppet-master!

You are a puppet!

In your instructor's opinion...

- › **Continuous Implementation / Continuous Delivery removes everything from our job that sucks!**
- › Software Developers used to only develop code. Ops team would deploy it
- › As DevOps Data Engineer you are responsible both for development and deployment
- › But deploying is incredibly annoying...



But you could be a puppet master!

In your instructor's opinion...

- › **Continuous Implementation / Continuous Delivery removes everything from our job that sucks!**
- › Software Developers used to only develop code. Ops team would deploy it
- › As DevOps Data Engineer you are responsible both for development and deployment
- › But deploying is incredibly annoying...



Continuous Integration Continuous Delivery

Rapid Delivery of increments

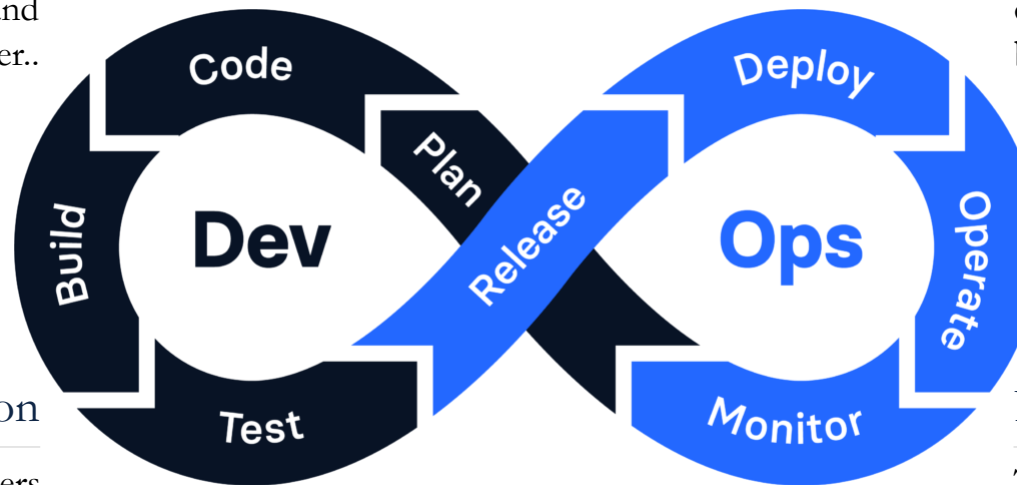
Continuous Delivery and Deployment enable teams to release updates to production quickly and reliably, ensuring that new features and fixes reach users faster..

Automated Testing

CI/CD integrates automated testing into the development process, ensuring that every code change is validated for quality and functionality before it is merged or deployed.

Frequent Integration

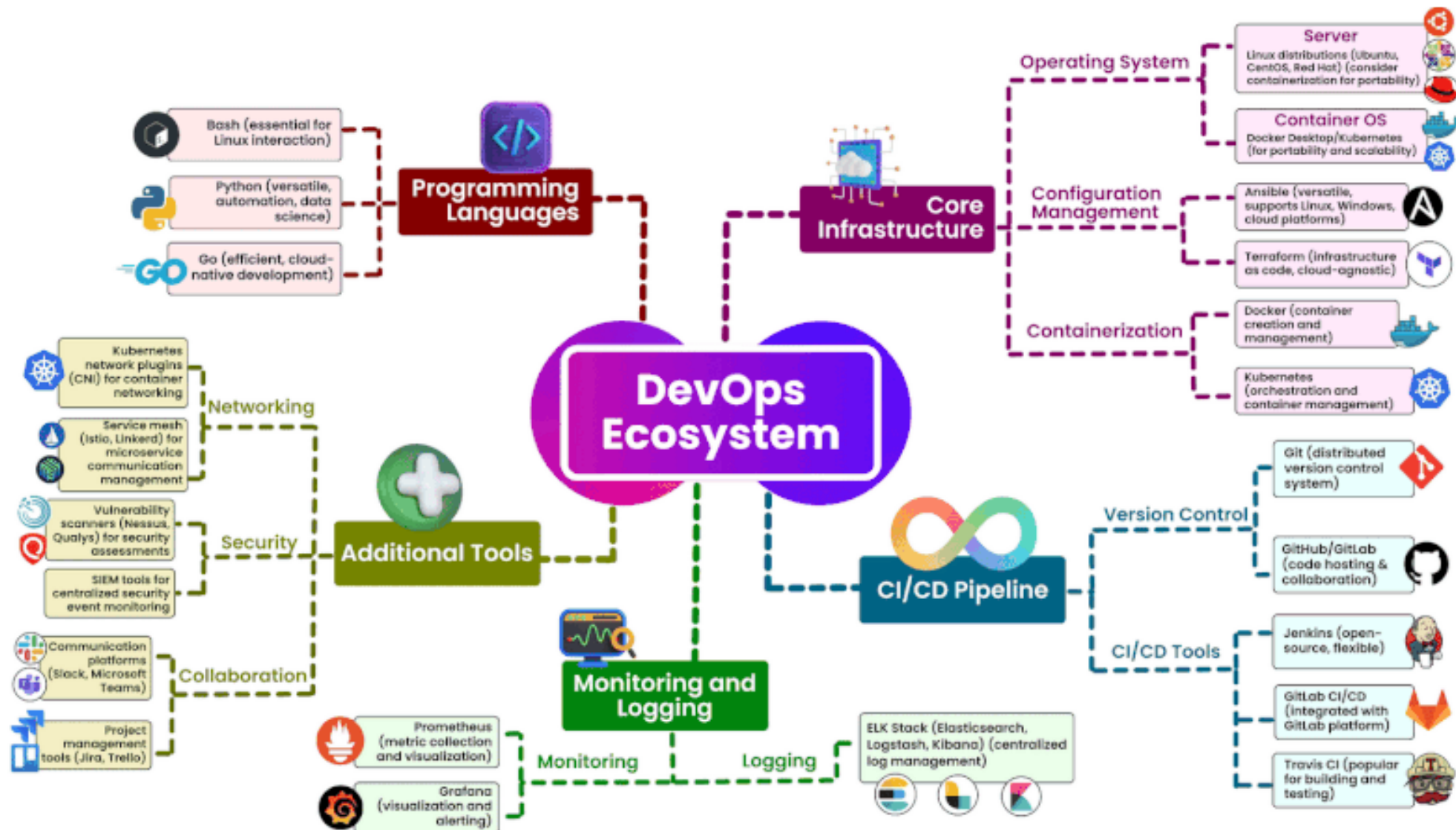
Continuous Integration encourages developers to frequently merge their changes into the main branch, allowing teams to catch and address conflicts or bugs early.



Pipeline Automation

The CI/CD pipeline automates the steps from code integration to deployment, reducing manual intervention and minimizing human error in the release process.

Continuous Integration Continuous Delivery



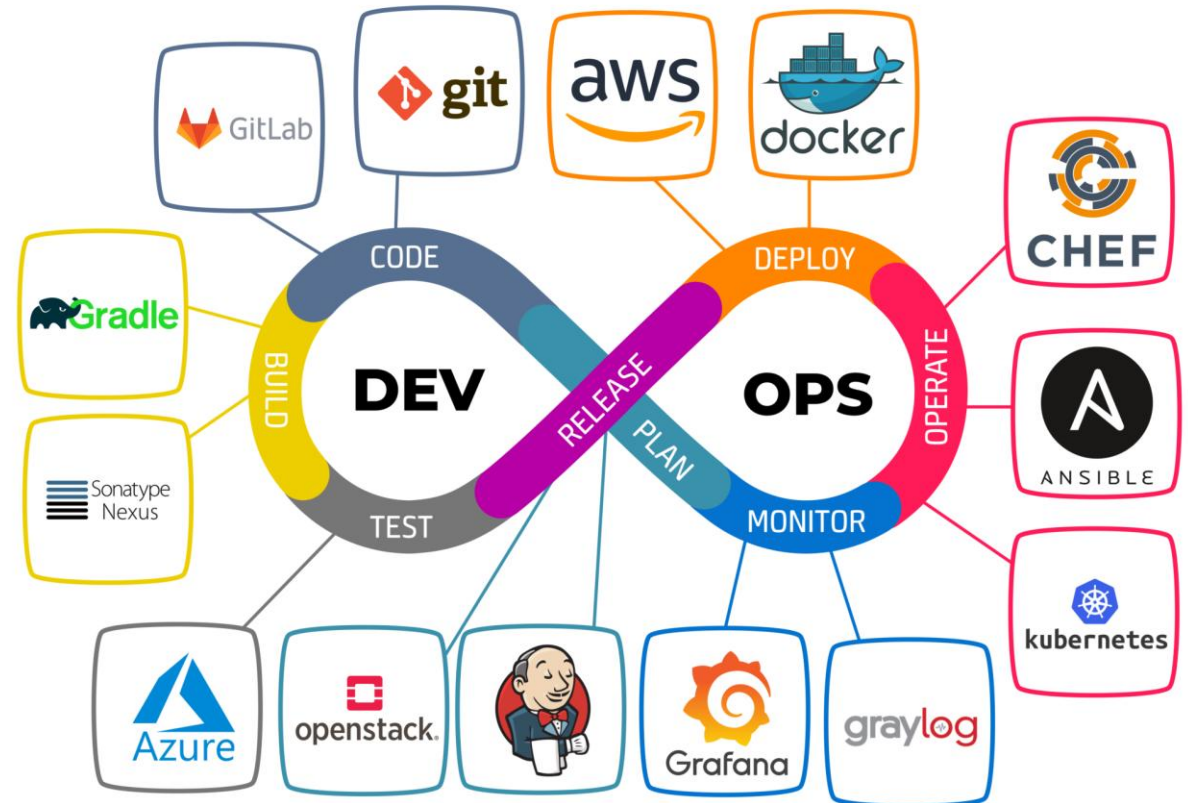
Source:
<https://ubiqware.net/en/our-approach-to-devops/>

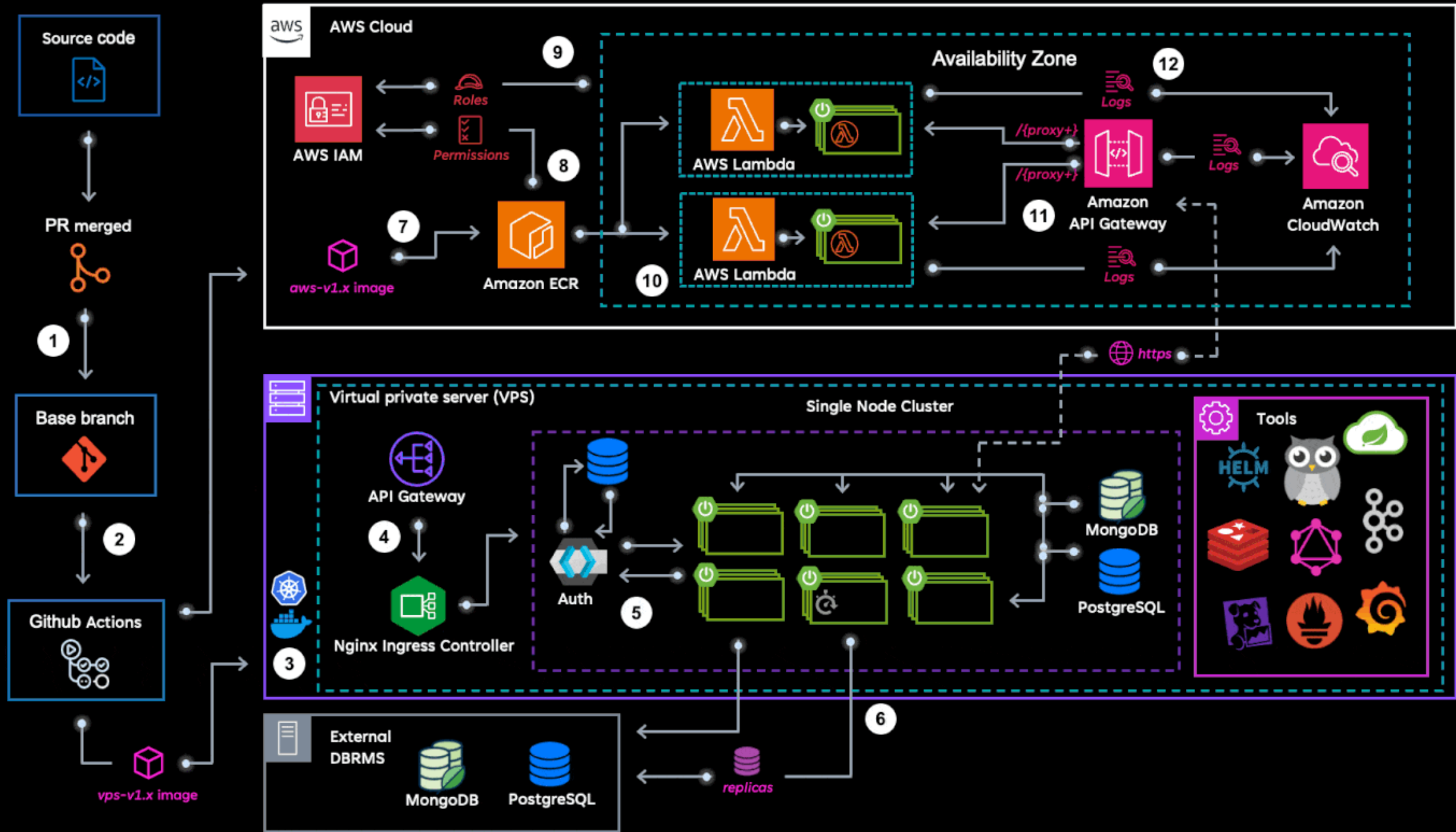
The DevOps tools jungle is growing by the day. Learn one tool per type, it will be simple to switch!

Don't get lost in the tool's jungle...

- › Learn only one tool per “type”
 - › Master one, other ones will be similar (e.g. switching from github to gitlab)
- › Most important types:
 - › Code Version Control Systems, Git (e.g. Gitlab)
 - › Containerization (e.g. Docker+Kubernetes)
 - › Monitoring (e.g. datadog)
 - › Cloud (e.g. AWS)

Tools, tools, tools,....





How long can you work on making a routine task more efficient before you're spending more time than you save?

CI/CD is about automating the boring stuff

- › Automatically **executing tests** (e.g., unit tests / integration tests) and code quality checks (e.g., sonarcube)
- › Automate **reporting** of test results (e.g., Jira Xray integration)
- › Automate **build** process + **store** builds/images/packages on artifactory
- › Automate **deployment** process

To save 1s on a task you do 50x/day, you can spend one day (times the number of people doing this)

HOW MUCH TIME YOU SHAVE OFF

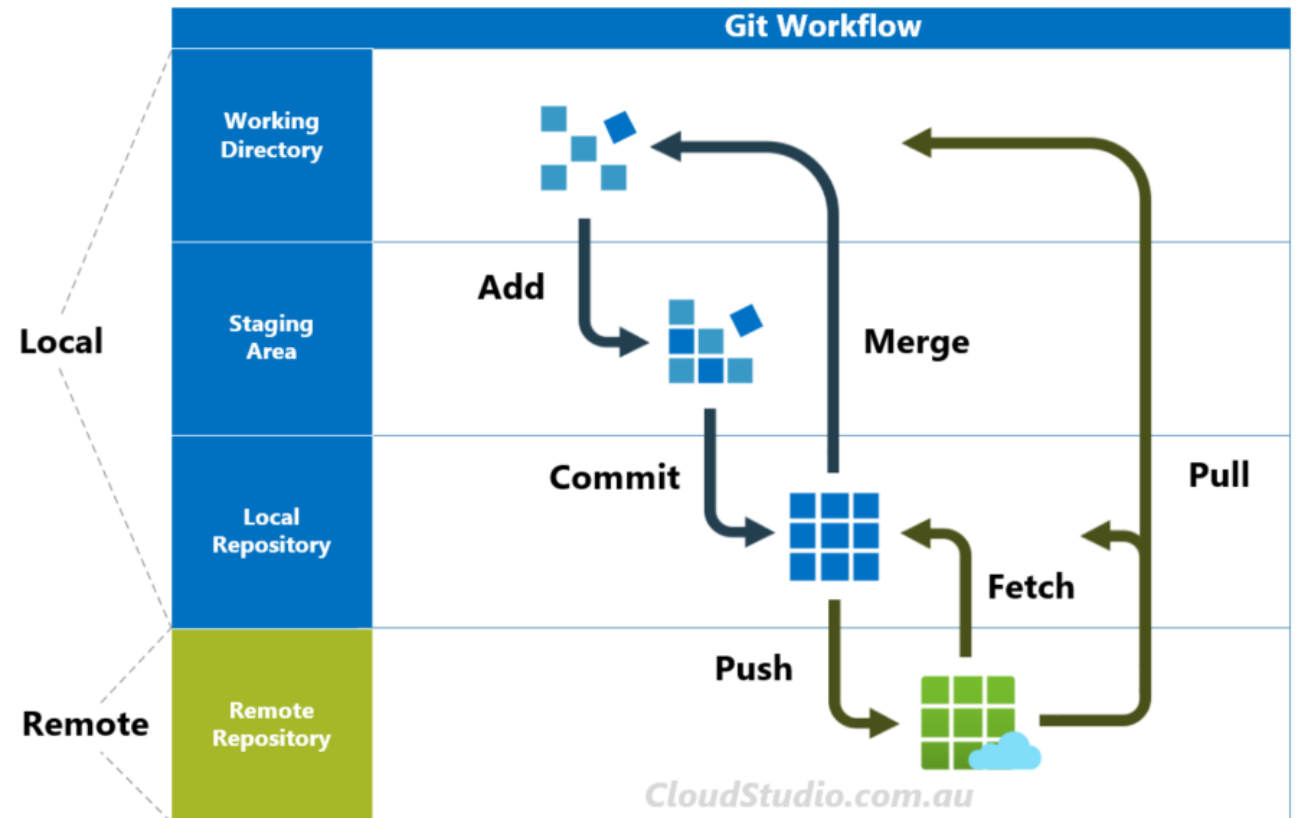
	HOW OFTEN YOU DO THE TASK					
	50/DAY	5/DAY	DAILY	WEEKLY	MONTHLY	YEARLY
1 SECOND	1 DAY	2 HOURS	30 MINUTES	4 MINUTES	1 MINUTE	5 SECONDS
5 SECONDS	5 DAYS	12 HOURS	2 HOURS	21 MINUTES	5 MINUTES	25 SECONDS
30 SECONDS	4 WEEKS	3 DAYS	12 HOURS	2 HOURS	30 MINUTES	2 MINUTES
1 MINUTE	8 WEEKS	6 DAYS	1 DAY	4 HOURS	1 HOUR	5 MINUTES
5 MINUTES	9 MONTHS	4 WEEKS	6 DAYS	21 HOURS	5 HOURS	25 MINUTES
30 MINUTES		6 MONTHS	5 WEEKS	5 DAYS	1 DAY	2 HOURS
1 HOUR		10 MONTHS	2 MONTHS	10 DAYS	2 DAYS	5 HOURS
6 HOURS				2 MONTHS	2 WEEKS	1 DAY
1 DAY					8 WEEKS	5 DAYS

A Git system is at the core of every CI/CD process because it serves as the source of truth and a central repository for the code, configuration, and history of the project.

Any Git will do, we focus on GitLab

A Git system like GitLab is central to CI/CD because it serves as the single **source of truth** for code and manages the **automation of development, testing, and deployment**. In CI, every code push triggers **pipelines** that run tests and builds, ensuring integration with the main codebase. GitLab merge requests facilitate code reviews, and successful tests enable **continuous deployment** to staging or production environments. Pipelines are managed via **.gitlab-ci.yml**, with automated feedback, security scans, and integration with other DevOps tools. This makes GitLab crucial for ensuring smooth, reliable, and efficient software delivery.

Reminder: fetch, pull, add, commit, push 😊

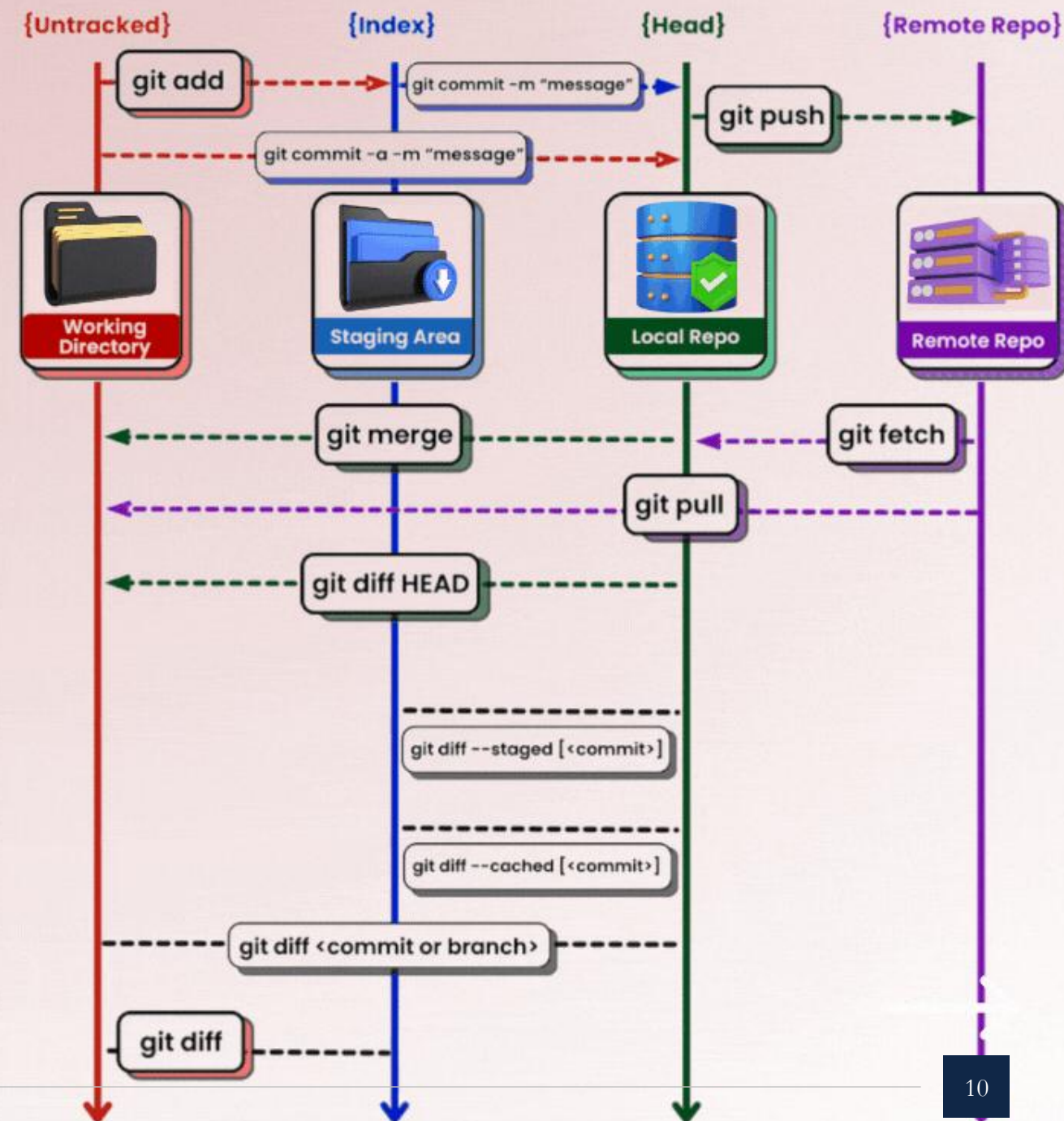


Master the Git workflow if you have not already. The below is not negotiable!

Reminder about the main commands:

- › **git add**: Stages changes (files) to be included in the next commit.
- › **git commit**: Records staged changes in the repository's history with a message.
- › **git push**: Uploads commits from your local repository to a remote repository.
- › **git merge**: Combines changes from one branch into another.
- › **git fetch**: Downloads changes from a remote repository but does not apply them.
- › **git pull**: Fetches and applies changes from a remote repository to your current branch (fetch + merge).

Source: Natan Hoart on LinkedIn
(<https://www.linkedin.com/in/natan-linked/>)



The pipeline mini graph always group jobs by stage, and display throughout GitLab when displaying pipeline or commit details

All1,000+FinishedBranchesTags				Clear runner cachesCI lintRun pipeline	
Filter pipelines				Show Pipeline ID	
Status	Pipeline	Created by	Stages		
Warning 02:24:47 2 days ago	Scheduled Ruby 3.2 ruby3_2 branch #1234145533 ruby3_2 000a47ea scheduled		✓ ✓ ✓ ✓ ✓ ✓ ! ✓ ✓ ✓ » → ✓ ✓ ✓ +30	▶ ↺ ↻ ⬇	
Passed 00:00:59 2 days ago	Merge branch 'generalize-ruby-sync' into 'rub... #1234144111 ruby-sync 6dc82a4d scheduledlatest		✓ »	⬇	
Failed 00:35:06 2 days ago	Ruby 3.1 MR [types: qa,code,rspec-predictive] #1234128996 147325 0bd7ba8a merged results		✓ ✓ ✓ ✓ ✓ ✓ ✗ ✓ » » » → ✓ ✓ ✓	▶ ↺ ↻ ⬇	

The pipeline details page displays a graph of all the jobs in the pipeline

Ruby 3.1 MR [types: docs]

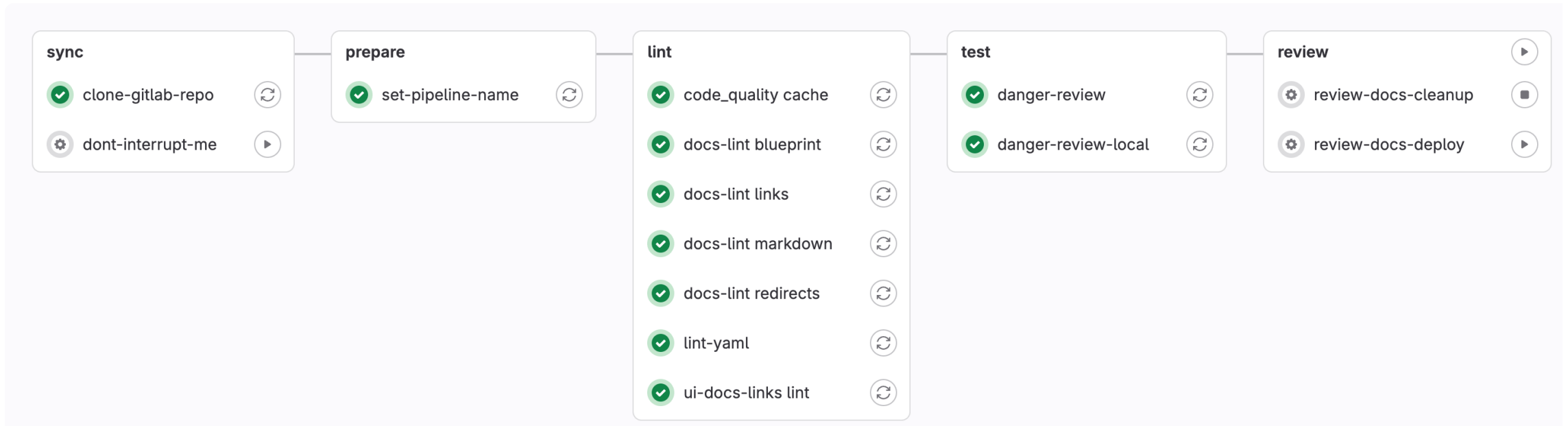
✓ Passed Marcel Amirault created pipeline for commit 8ea89d6f 17 minutes ago, finished 7 minutes ago

Related merge request !147744 to merge docs-to-setup-to-set-up into master

latest merged results 14 jobs 9 minutes 28 seconds, queued for 1 seconds

Pipeline Needs Jobs 14 Tests 0 Code Quality

Group jobs by Stage Job dependencies



When you configure jobs with the **needs** keyword, you have two options for how to group the jobs in the pipeline details page: **Stage**

Pipeline Needs Jobs 9 Tests 0

Group jobs by Stage Job dependencies

build

- ✓ build-job1 ↺
- ✓ build-job2 ↺
- ✓ build-job3 ↺

test

- ✓ lint-job ↺
- ✓ test-job1 ↺
- ✓ test-job2 ↺
- ✓ test-job3 ↺

deploy

- ✓ deploy-job1 ↺
- ✓ deploy-job2 ↺

When you configure jobs with the **needs** keyword, you have two options for how to group the jobs in the pipeline details page: [Dependencies](#)

Pipeline Needs Jobs 9 Tests 0

Group jobs by

Stage

Job dependencies

Show dependencies



Using the **needs** keyword you can also create jobs within a stage which are not executed in parallel

GitLab demo script

```
stages:  
  - test
```

```
unit-tests:  
  stage: test  
  script:  
    - echo "Running unit tests..."
```

```
report-test-results:  
  stage: test  
  script:  
    - echo "Reporting test results..."  
  needs:  
    - unit-tests
```

Using **needs** we can
model dependencies

Dependent jobs in a stage

Group jobs by

test

 report-test-results

 unit-tests





Pipeline Jobs 3 Tests 0

Group jobs by **Show dependencies** 

 unit-tests
test

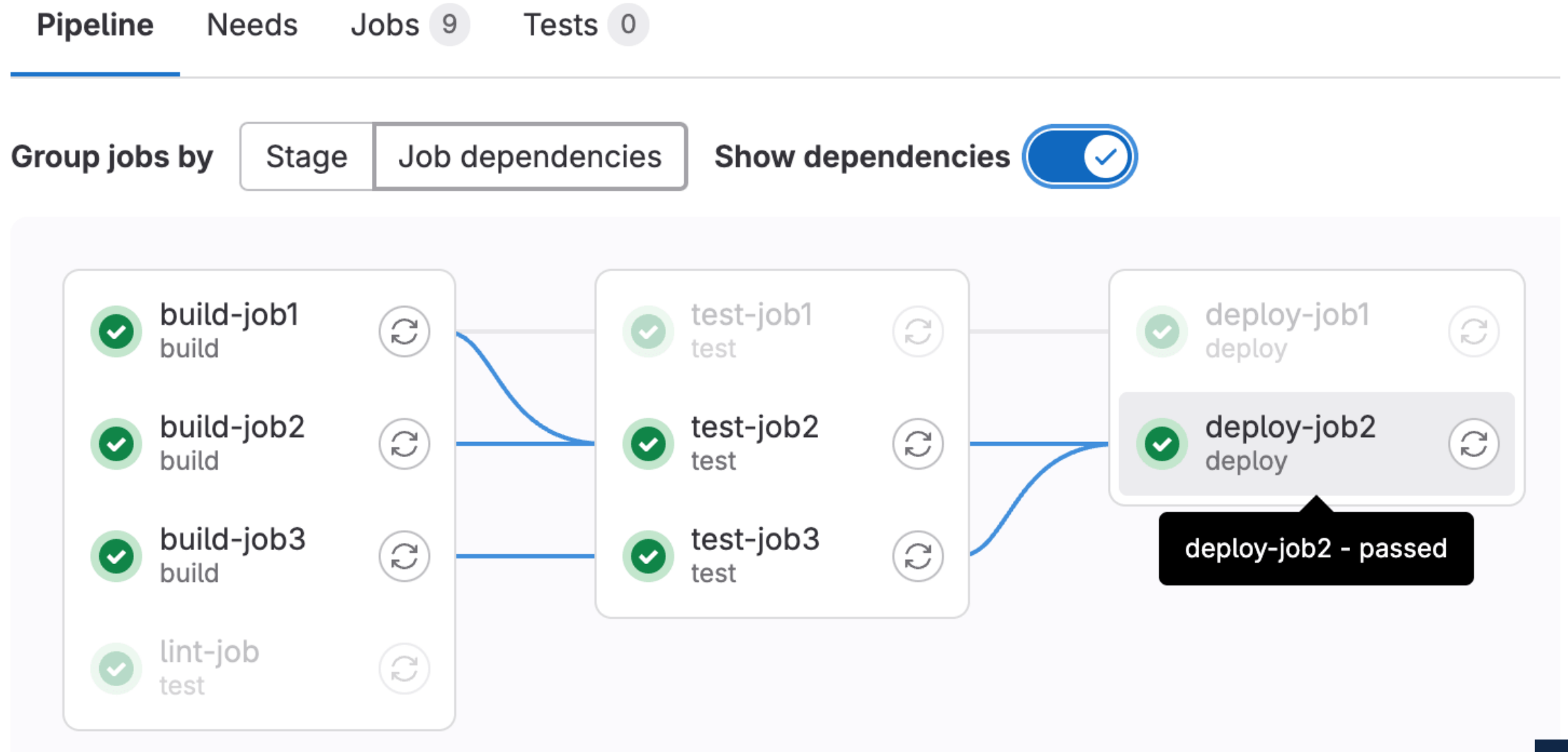


 report-test-results
test



Jobs in one stage are not
executed in parallel...

When you hover over a job in the **job dependencies** view, every job that must run before the selected job is highlighted



In GitLab CI/CD **dependencies** in jobs in a stage can be used to make jobs executed in a given order, **artifacts**, can be used to store results between different jobs/stages

You can point to another gitlab-ci.yml file

- › **Dependencies:** In GitLab CI, dependencies define which jobs from previous stages pass their artifacts to the current job. This ensures that jobs can access outputs (e.g., test reports) from earlier stages without re-executing them.
- › **Artifacts:** Artifacts are files or directories generated by a job that are stored and can be passed to subsequent jobs. They allow the output of one job (e.g., test results or build files) to be reused in later stages of the pipeline.

Example of stages

```
stages:
  - test
  - sonar

unit_tests:
  stage: test
  image: python:3.9
  script:
    - pip install pytest
    - pytest --junitxml=report.xml
  artifacts:
    paths:
      - report.xml
  only:
    - merge_requests

sonar_analysis:
  stage: test
  image: sonarsource/sonar-scanner-cli:latest
  dependencies:
    - unit_tests
  script:
    - sonar-scanner -Dsonar.python.coverage.reportPaths=report.xml \
  only:
    - merge_requests
```

report.xml created

report.xml stored

report.xml used

Makes second job dependent on first job

Installing components onto the runner on the fly can be slow & costly if executed often. A typical pattern is therefore to bundle dependencies into a docker image

Define all components in Docker...

```
FROM python:3.9-slim
ENV POETRY_VERSION=1.6.1 \
    POETRY_HOME="/opt/poetry" \
    POETRY_NO_INTERACTION=1

ENV PATH="$POETRY_HOME/bin:$PATH"
RUN apt-get update && \
    apt-get install -y curl && \
    curl -sSL
https://install.python-poetry.org
| python3 - && \
    apt-get clean && rm -rf
/var/lib/apt/lists/*

COPY pyproject.toml poetry.lock*
/app/
WORKDIR /app
RUN poetry install --no-root
CMD ["python", "--version"]
```

...CI/CD pipeline to push image...

```
image: docker:24.0.1

stages:
  - build
  - push

build_docker_image:
  stage: build
  script:
    - docker build -t
$CI_REGISTRY_IMAGE:latest

push_docker_image_to_artifactory:
  stage: push
  script:
    - docker push
$ARTIFACTORY_URL/docker-
virtual/.../my-image:latest
  only:
    - main
```

Publishing to
artifactory

... use that image in another pipeline

```
image: artifactory/.../my_image

stages:
  - test
  - deploy

unit_test_job:
  stage: test
  script:
    - echo "Running unit tests"
    - pytest

deploy_job:
  stage: deploy
  script:
    - echo "Deploying application"
    - ./deploy.sh
  rules:
    - if: '$CI_COMMIT_BRANCH ==
"master"'
```

the image we build before

Another common pattern is to have a dedicated CI/CD repository containing pipelines and application repositories “pointing” to the CI/CD repo

You can point to another gitlab-ci.yml file

- › One repository per dataset/source application, for example spark application code
- › In Settings → CI/CD → General pipelines we can point to another pipeline:

CI/CD configuration file

group/another-repo/.gitlab-ci.yml

Trigger a pipeline located in
another repo

Alternatively, trigger a pipeline from another repo

```
stages:
  - build
  - test
  - trigger

build_job:
  ...

test_job:
  ...

trigger_another_repo_pipeline:
  stage: trigger
  trigger:
    project: "group/another-repo"
    branch: "main"
  only:
    - main
```

GitLab Runners are a critical component of GitLab's CI/CD system. They function as isolated environments that execute jobs defined in GitLab CI pipelines.

Runners are flexible, scalable and customizable

Runners support **various execution environments**, including Docker, Shell, and Kubernetes, making them adaptable to a wide range of project requirements. They can handle **concurrent job executions**, significantly improving pipeline efficiency and reducing total execution time. There are three types of runners:

- › **Shared Runners:** Offered by GitLab for use across multiple projects, typically maintained by GitLab itself.
- › **Specific Runners:** Configured for use within particular projects, granting more control over the environment.
- › **Group Runners:** Assigned to multiple projects within a GitLab group, providing flexibility across related repositories.

An example of instance runners

Instance runners

These runners are available to all groups and projects.

Each CI/CD job runs on a separate, isolated virtual machine.

Enable instance runners for this project



Available instance runners: 102

● #11573930 (KzYhZxBvi)

1-blue.shared-gitlab-org.runners-manager.gitlab.com

gitlab-org

● #11573990 (NL4gfoBem)

2-blue.shared-gitlab-org.runners-manager.gitlab.com

gitlab-org

● #11574038 (sUrYYgEG8)

3-blue.shared-gitlab-org.runners-manager.gitlab.com

gitlab-org

Common pattern: Custom runners --- You can run the GitLab Daemon on any machine to have custom runners for a repository or a group (=set of repositories)

Custom runners make it easy to interact with your cloud

In a production grade setup, you typically use gitlab runners running on your own infrastructure (e.g. AWS EC2). Benefits include:

- › **Scalability**: You can create as many runners as you need to break bottlenecks in unittesting / deployments
- › **Customizability**: If your Gitlab Runner runs on EC2, you can assign an IAM role to that EC2 to e.g., be able to write to S3 (e.g. to deploy spark application code for a glue job)
- › On Gitlab → CICD → Runners → New project runner to receive a token to register any machine. Register using:

```
gitlab-runner register
--url https://gitlab.com
--token glrt-t3_K5hUYyHyokAzH9g4d5bdv64d8vr9pL
```

An example of custom runners

Project runners

These runners are assigned to this project.

New project runner



Register "myrunner" runner

Platform

Operating systems



Linux



macOS



Windows

Cloud



Google Cloud

GitLab variables are key-value pairs that can be used to store configuration values, sensitive information, or any other data needed by CI/CD pipelines

There are custom and predefined variables

Variables are used in the pipeline using the `$` accessor, e.g., `docker build -t $CI_REGISTRY_IMAGE:latest`.

Custom variables can be created from the GitLab Web UI:

- › They have a **key**, which is used inside yaml scripts. Common practice is to use uppercase notation, e.g., `MY_VAR`
- › They have a **value**, which the key points to

There are also some **predefined variables**:

- › `CI_COMMIT_REF_NAME`: Contains the branch or tag name for which the pipeline is running.
- › `CI_PIPELINE_SOURCE`: Indicates the source of the pipeline, such as `push`, `merge_request_event`, `schedule`, or `webhook`, among others.

Setting a variable from GitLab WebUI

Description (optional)

some description

The description of the variable's value or usage.

Key

MY_VAR

You can use CI/CD variables with the same name in different places, but the variables might overwrite each other. [What is the order of precedence for variables?](#)

Value

hfjdk(#&KLJRE&#jdk(*JDK@



Custom variables can be masked, hidden and protected to store secrets safely

There are three types of variables:

- › **Visible:** Can be seen in job logs.
- › **Masked:** Masked in job logs but value can be revealed in CI/CD settings. Requires values to meet regular expressions requirements.
- › **Masked and hidden:** Masked in job logs, and can never be revealed in the CI/CD settings after the variable is saved.
- › **Protected flag** will make only **protected branches** being able to use the variable (e.g., main, release branch)
- › Disadvantage: Secrets are not created using IaC + GitLab might **not be safe enough** to store company secrets (e.g., username/password for PROD database)

Example of two variables in Gitlab CI/CD

Jannic Cutura / Demo Project / CI/CD Settings

Variables can have several attributes. [Learn more.](#)

- **Visibility:** Set the visibility level for the value. Can be visible, masked, or masked and hidden.
- **Flags**
 - **Protected:** Only exposed to protected branches or protected tags.
 - **Expanded:** Variables with `$` will be treated as the start of a reference to another variable.

CI/CD Variables </> 2		Reveal values	Add variable
Key ↑	Value	Environments	Actions
MY_OHTER_VAR  Expanded	***** 	All (default) 	 
MY_VAR  a variable Protected Masked	***** 	All (default) 	 

As your project grows, the `.gitlab-ci.yml` file can become lengthy/hard to manage. GitLab allows you to split this file into multiple files using the `include` keyword.

Choosing how to split content into multiple files by...

Pipeline Stage: Separate files for each stage (e.g., build, test, deploy); When stages are complex and contain multiple jobs; Easier to focus on one stage at a time.

Microservice or Component: Separate files for different services or components in a monorepo; When working with microservices or modular components; Teams can work independently on their components.

Environment: Separate files for different deployment environments (e.g., development, staging, production); When environments have distinct configurations; Clear separation of environment-specific settings.

Job Type or Purpose: Separate files for variables, templates, and specific job types; For complex pipelines with reusable configurations; Promotes reusability and reduces duplication.

Team Ownership: Separate files based on team responsibilities; In large organizations with multiple teams collaborating on a project; Teams can manage their CI/CD configurations independently.

Repository structure + gitlab-ci.yml

```
project-root/  
├── .gitlab-ci.yml  
└── ci/  
    ├── variables.yml  
    ├── build.yml  
    ├── test.yml  
    └── deploy.yml
```

```
# .gitlab-ci.yml  
include:  
  - 'ci/variables.yml'  
  - 'ci/build.yml'  
  - 'ci/test.yml'  
  - 'ci/deploy.yml'  
  
stages:  
  - build  
  - test  
  - deploy
```